

Advanced Iceberg

CDC with MERGE

Lesson objectives

Advanced Iceberg: CDC with MERGE

1. Define Change Data Capture (CDC) and its importance in data systems.
2. Define Slowly Changing Dimensions (SCD) and compare it with CDC.
3. Explain the syntax of the `MERGE` statement and how it relates to CDC and SCD.
4. Demonstrate use of the `MERGE` statement in Iceberg.

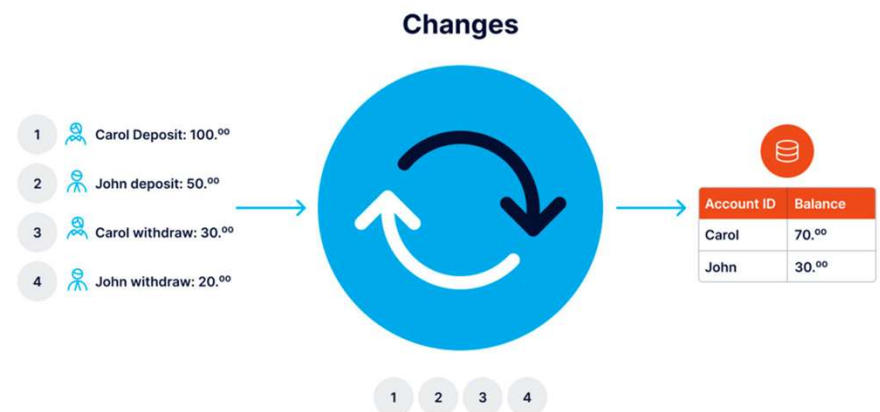
CDC defined

Wikipedia defines [CDC](#) as:

In databases, change data capture (CDC) is a set of software design patterns used to determine and track the data that has changed so that action can be taken using the changed data.

...

CDC occurs often in data-warehouse environments since capturing and preserving the state of data across time is one of the core functions of a data warehouse, but CDC can be utilized in any database or data repository system.



CDC source methods

METHOD	DESCRIPTION
Audit columns	By using existing “LAST_UPDATED” or “DATE_MODIFIED” columns you can create a CDC solution by persisting the last date used and leveraging it in a simple query.
Table deltas	A list of changes could be created by using a ‘tablediff’ utility to compare the source and target tables.
Table triggers	Created table-based triggers can run on inserts, updates and deletes to produce an ordered list of activities.
Transaction logs	The transaction logs (aka redo logs) that databases create for durability, produce an inherent ordered list of activities without additional effort.

CDC vs change tracking

Change tracking is a subset of CDC. It allows the net changes made to data to be returned from a query easily. It can let you know a given row has changed since your last query, but does not have an idea about:

- The different value changes that have been made to the row, or
- The number of times it was changed

This approach usually syncs up the destination table on a regular schedule of the current state of any records in the source table that have changed since the last check.

Primarily produced from the “audit columns” and “table deltas” methods.

What is SCD and how is it different than CDC?

CDC identifies and tackles the data that has been changed.

A Slowly Changing Dimension (SCD) is a dimension table that manages & stores both historical & current data in a data warehouse or lakehouse.

Multiple approaches exist, but the most popular are:

- **Type 1** – Only maintains current data; no historical data is kept
- **Type 2** – Keeps current as well as historical data in the table made to the row

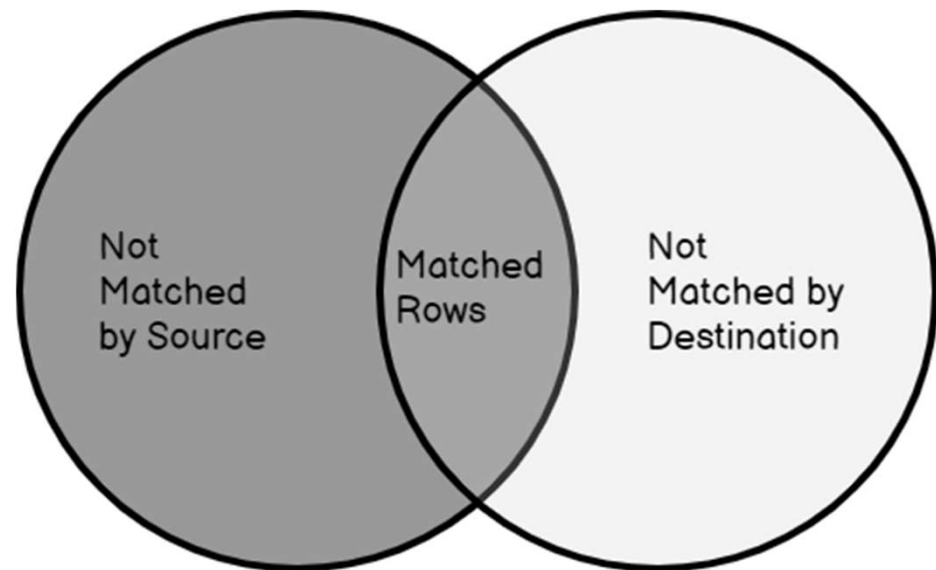
MERGE **to the rescue**

The `MERGE` SQL statement allows...

the deltas created from a CDC process to be...

integrated/merged into a target table leveraging...

various SCD strategies



MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

where when_clause is one of

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```


MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

The replicated
(dest) table



where when_clause **is one of**

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

← The delta
(changes) table

where when_clause **is one of**

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

An expression that
returns a boolean
value

where when_clause **is one of**

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

MERGE syntax

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...] ←
```

where when_clause **is one of** ←

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

Multiple when clauses are allowed, but stops checking once one is met

MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

where when_clause **is one of**

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

Multiple when
clauses allowed
for each of these
three options

MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

where when_clause **is one of**

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

WHEN MATCHED
clauses support
UPDATE or
DELETE actions

MERGE **syntax**

```
MERGE INTO target_table [ [ AS ] target_alias ]  
USING { source_table | query } [ [ AS ] source_alias ]  
ON search_condition  
when_clause [...]
```

where when_clause **is one of**

```
WHEN MATCHED [ AND condition ]  
    THEN DELETE
```

```
WHEN MATCHED [ AND condition ]  
    THEN UPDATE SET ( column = expression [, ...] )
```

```
WHEN NOT MATCHED [ AND condition ]  
    THEN INSERT [ column_list ] VALUES (expression, ...)
```

WHEN NOT
MATCHED **clauses**
ONLY support
INSERT **actions**

MERGE implementation

Conceptual wrapper of multiple `INSERT`, `DELETE` & `UPDATE` statements

- **Write-time**
 - 1+ data files added for any newly inserted rows
 - 1+ delete files are added for any rows to be completely deleted
 - 1+ delete files are added for any rows being updated
 - 1+ data files added to include the full replacements for any rows being updated
- **Read-time**
 - Same as prior snapshot
 - Then deletes rows identified in delete files
 - Adds newly inserted & updated rows in new data files

Metadata is modified via the previously presented generic DML operation flow

Instructor demonstration

Using MERGE (15 mins)

Hands-on exercise

Utilize Iceberg's `MERGE` statement
(30 mins)

Lesson summary

Advanced Iceberg: CDC with `MERGE`

1. Change Data Capture (CDC) is a set of design principles to help recognize changing datasets and manage those changes effectively.
2. A Slowly Changing Dimension (SCD) is a type of table that manages and stores both historical and current data over time in a data warehouse or lakehouse.
3. The `MERGE` statement allows the changes created from a CDC process to be integrated into a target table using various SDC strategies.

Advanced Iceberg

Schema & partition evolution

Lesson objectives

Advanced Iceberg: Schema & partition evolution

1. Describe how schema evolution functions as an Iceberg feature.
2. Explain how Iceberg uses hidden partitioning.
3. Describe the transformation function that allows a partition to be based on a derivative of a timestamp column.
4. Understand the limitations of partition evolution.
5. Demonstrate how to change Iceberg partition configuration.

Schema evolution

In addition to simple operations such as renaming a table, Iceberg supports:

- ADD – add a new column
- DROP – remove an existing column
- RENAME – rename an existing column
- UPDATE – widen the type of a column
- REORDER – change the order of columns

Schema updates are *metadata changes* – no data files are rewritten.



Schema evolution correctness

Iceberg guarantees that schema evolution changes are independent and free of side-effects, without rewriting files.

Iceberg uses unique IDs to track each column in a table. When you add a column, it is assigned a new ID so existing data is never used by mistake.



Schema evolution examples

```
ALTER TABLE planes RENAME COLUMN name TO description;
```

```
ALTER TABLE planes ADD COLUMN year integer;
```

```
ALTER TABLE planes DROP COLUMN color;
```


Iceberg's hidden partitioning

Other table formats like Hive support partitioning, but Iceberg supports *hidden partitioning*.

- Iceberg handles the tedious and error-prone task of producing partition values for rows in a table.
- Iceberg avoids reading unnecessary partitions automatically. Consumers don't need to know how the table is partitioned or need to add extra filters to their queries.
- Iceberg partition layouts can evolve as needed.

Partitioning based on transforms

Like Hive, Iceberg allow for partitions based on one, or more, columns (which do NOT have to be last in the table definition), but also feature timestamp-based transformations using an existing column without needed to create a new column using one of the following possibilities:

- **year(ts)** – A partition is created for each year.
- **month(ts)** – A partition created for each month of the year.
- **day(ts)** – A partition is created for each day of each year.
- **hour(ts)** – A partition is created for each hour of each day.

Partition evolution examples

```
CREATE TABLE orders_iceberg
WITH (partitioning =
      ARRAY[ 'month(orderdate) ' ]) AS
SELECT * FROM tpch.sf1.orders;

ALTER TABLE orders_iceberg
SET PROPERTIES partitioning =
      ARRAY[ 'day(orderdate) ' ];
```

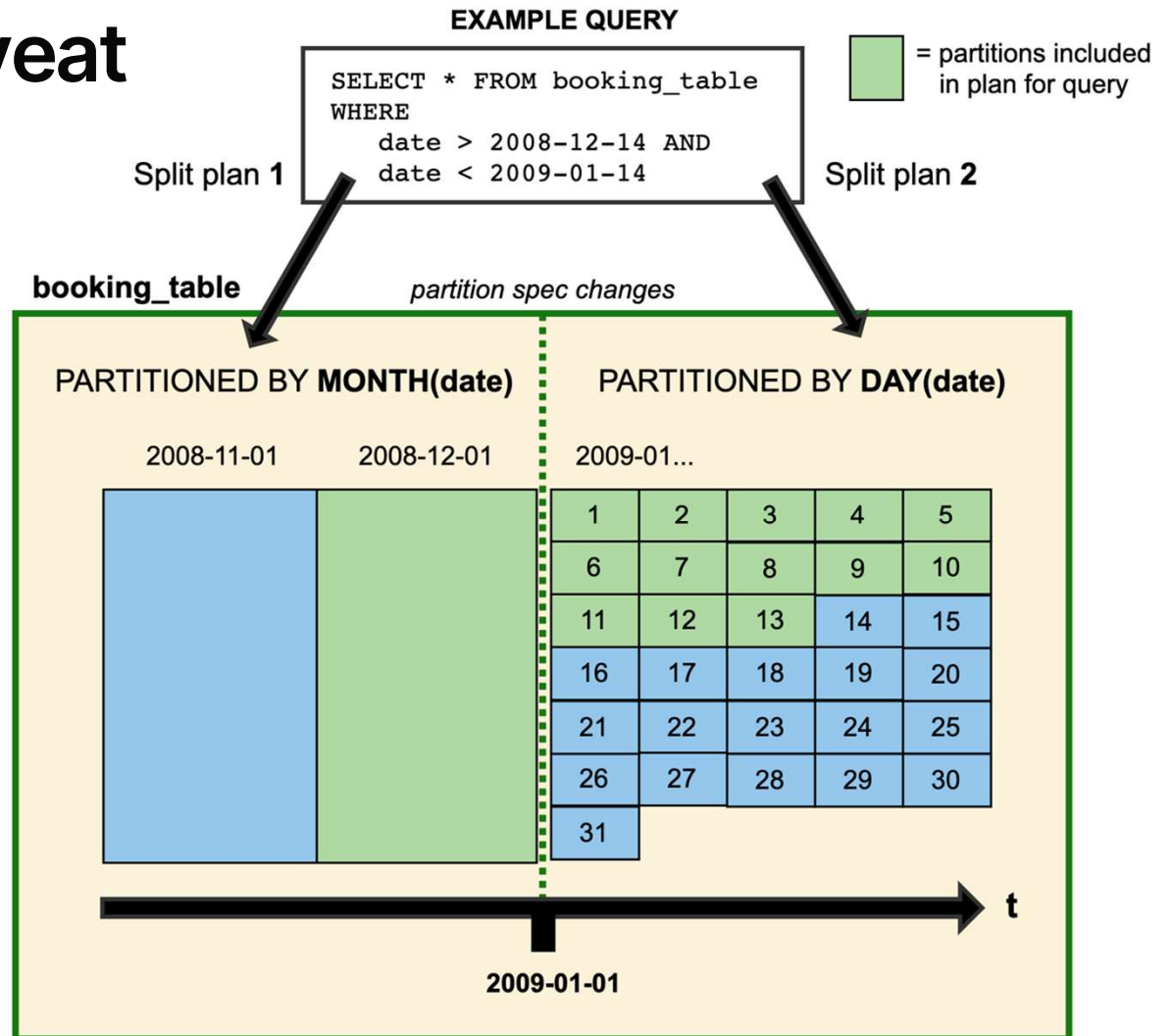
📁	orderdate_day=1996-06-13/
📁	orderdate_day=1996-09-26/
📁	orderdate_day=1997-11-04/
📁	orderdate_month=1992-01/
📁	orderdate_month=1992-02/
📁	orderdate_month=1992-03/
📁	orderdate_month=1992-04/
📁	orderdate_month=1992-05/

Partition change caveat

When you change the partitioning strategy, the existing data is NOT rewritten.

New data is written using the new layout.

Iceberg and Starburst will read only what they need to.



Instructor demonstration

Change Iceberg partitions (10 mins)

Lesson summary

Advanced Iceberg: Schema & partition evolution

1. Iceberg includes support for schema evolution, including add, drop, rename, update, and reorder operations.
2. This ensures that changes to schema do not impact the accuracy of your data or require rewrites.
3. Iceberg uses hidden partitions to automatically create row-level partitions without need for user intervention.
4. Iceberg allows for partitions based on transformations of timestamps.
5. Changing the partition strategy does not rewrite the data. This has both advantages and disadvantages.

Advanced Iceberg

Snapshots & compaction

Lesson objectives

Advanced Iceberg: Snapshots & compaction

1. Describe why maintenance activities are needed when using Iceberg.
2. Explain how to expire older/unwanted snapshots.
3. Describe how to remove orphaned files in Iceberg.
4. Explain how to compact files in Iceberg.
5. Demonstrate how snapshots work in Iceberg.

Data and metadata sprawl

Short ingestion cycles (to include real-time) plus DML operations create potential longer-term issues with the Iceberg table format such as:

- A large amount of metadata for all of the snapshots being created
- Data files that are being referenced only by older snapshots that cannot be deleted
- Very likely, a high percentage of new files being created fall into the “small files problem”

Required maintenance

To address the data/metadata sprawl, the following actions need to occur periodically:

- **Compact files** – Rebuild small files into a fewer number of larger files
 - Critical activity to reduce the small files problems
 - Reconciles delete files to improve performance by eliminating the need to merge during read
- **Expire snapshots** – Shorten the history of available snapshots
 - Removes the snapshot information from the metadata files
 - No longer available for time-travel queries
 - Only deletes data files that are no longer referenced by remaining snapshots
- **Remove orphaned files** – Deletes any files present that are not referenced by any existing snapshot which may be present due to
 - Job failures from Starburst and possibly other distributed engines
 - Any problems with snapshot expirations

Compacting files

The optimize command uses an optional `file_size_threshold` parameter (default is 100MB) to determine the files to focus on compacting into larger ones.

For example, the following would focus on files under 100 megabytes in size to be merged into a few number of larger files.

```
ALTER TABLE orders_iceberg EXECUTE optimize;
```

This next example targets files in the table that are under 10 megabytes in size.

```
ALTER TABLE orders_iceberg  
    EXECUTE optimize(file_size_threshold => '10MB');
```

Additional compaction options

Use a `WHERE` clause to target metadata about data files or the partition strategy.

For example, the following only compacts files that have arrived recently.

```
ALTER TABLE orders_iceberg EXECUTE optimize  
    WHERE $file_modified_time > <yesterday>;
```

This next example targets a particular partition.

```
ALTER TABLE orders_iceberg EXECUTE optimize  
    WHERE partition_key = <value>;
```

Expiring snapshots

The expire snapshots command uses a `retention_threshold` parameter to determine which snapshots and related metadata and data files to delete.

For example, the following would remove snapshots older than 7 days.

```
ALTER TABLE orders_iceberg
```

```
    EXECUTE expire_snapshots (retention_threshold => '7d');
```

Removing orphaned files

The remove_orphaned_files command also uses `retention_threshold` to delete any orphaned data files that are or older than a specific age.

For example, the following would remove orphaned files older than 7 days.

```
ALTER TABLE orders_iceberg
```

```
EXECUTE
```

```
  remove_orphan_files(retention_threshold => '7d');
```

Hands-on exercise

Exercise advanced features of
Iceberg (30 mins)

Lesson summary

Advanced Iceberg: Snapshots & compaction

1. Frequent ingestions and DML operations create a large amount of metadata, which can present problems as they grow larger.
2. To help keep metadata in check, periodic maintenance is required, including: expiring snapshots, removing orphaned files, and compacting files.
3. Users should periodically run an ALTER TABLE command to clear all snapshots beyond a certain age.
4. Orphaned files can also be removed using ALTER TABLE.
5. Small files can be located and merged using Iceberg, a process known as compaction. This improves performance.